

# Deep Reinforcement Learning-Based Control of Bicycle Robots on Rough Terrain

1<sup>st</sup> Xianjin Zhu

School of Mechatronics Engineering  
Harbin Institute of Technology  
Harbin, China  
19B308005@stu.hit.edu.cn

2<sup>nd</sup> Xudong Zheng

School of Automation  
Beijing University of Posts and Telecommunications  
Beijing, China  
xudongzheng\_buaa@163.com

3<sup>rd</sup> Yang Deng

Department of Automation  
Tsinghua University  
Beijing, China  
dengyang@mail.tsinghua.edu.cn

4<sup>th</sup> Zhang Chen

Department of Automation  
Tsinghua University  
Beijing, China  
cz\_da@tsinghua.edu.cn

5<sup>th</sup> Bin Liang

Department of Automation  
Tsinghua University  
Beijing, China  
bliang@tsinghua.edu.cn

6<sup>th</sup> Yu Liu\*

School of Mechatronics Engineering  
Harbin Institute of Technology  
Harbin, China  
lyu11@hit.edu.cn

**Abstract**—The bicycle robot is a kind of unmanned mobile robot with great potentials. However, the control of such robots on rough terrain under model uncertainties and disturbances is challenging due to the lateral instability and underactuated characteristics. In this paper, a controller based on deep reinforcement learning and Stanley algorithm is designed to achieve the path tracking and balancing control for a bicycle robot on rough terrain. First, the outputs of the controller are compensated by residual reinforcement learning to reduce the gap between the training environment and the test environment. Then, in balancing control, the piecewise curriculum learning is designed to accelerate the training process. In path tracking control, Stanley algorithm is used. The proposed algorithm is tested in Gazebo simulation environment. The results showed that the proposed algorithm achieved good performance.

**Index Terms**—bicycle robot, path tracking, balancing control, reinforcement learning, curriculum learning

## I. INTRODUCTION

Bicycle robots (BRs) have many characteristics, such as small in size, light in weight, compact in structure, flexible in motion, and they have great application potentials in jungle exploration and mountain search and rescue [1]. BR is an underactuated and nonholonomic system with complex dynamic characteristics. Its two wheels are arranged longitudinally and it has the characteristic of lateral instability [2], [3]. All these make it widely concerned in the robotics field.

The research community has focused on the motion control of BR in the past decades. Balancing control based on classical control theory has been widely studied [4]–[7]. Moreover, the coupling of balance control and path tracking has attracted the attention of many researchers [8], [9]. Even though the motion control of BR has been studied extensively and better control performance has been achieved on flat road, there are some limitation for the motion control of BR on rough terrain, which is mainly shown in that the coupling BR at different speeds,

steering angles and pitching angles on rough terrain makes the dynamics characteristics more complex [10]. In this way, the difficulty and workload of establishing accurate analytical dynamics models increases dramatically. At the same time, it also restricts the performance of the controller.

The deep reinforcement learning (DRL) has obvious advantages in solving the aforementioned problems caused by the traditional methods [11], [12]. RL usually formalizes a sequential decision problem with Markov Decision Process (MDP), which considers how an agent should behave in its environment over time to maximize a scalar reward signal. This approach usually relies on trial and error to optimize the agent rather than deducing controller parameters based on analytic dynamics models. Some researchers have tried to introduce DRL algorithm to balance control and path tracking problems of BR on flat road, and achieved some outcomes [13]–[15].

However, designing DRL-based control method for BR is challenging since the BR not only has a higher state dimension on rough terrain than on flat road, but also has a more prominent problem of sparse reward. These make the training of end-to-end DRL slowly, or even hard to converge. In this paper, we construct a task set based on the balancing control task corresponding to the different subset of state space of BR under different speeds, steering angles and pitch angles. The curriculum learning [16]–[18] is a method for speeding the training of the DRL. By designing a task proposal module and an agent pool, the piecewise curriculum learning (PCL) is constructed to effectively train the balancing control task set. In addition, a framework of residual reinforcement learning (RRL) [15] which is used to compensate the bias output of the balancing controller and path tracking controller caused by the gap of the training model and the target model.

The rest of this paper is structured as follows. Section II describes the system overview and the construction of the simulation environment. Section III gives an overview of

This work was supported by the National Natural Science Foundation of China 62203252, 52205008.

algorithms proposed. Simulation results are given in Section IV, followed by a conclusion in Section V.

## II. SYSTEM OVERVIEW

### A. Bicycle Robot

The BR with an inertia wheel which is selected as the research object consists of five bodies, namely, front wheel, rear wheel body frame, steering handle and inertia wheel, as shown in Fig. 1 and Fig. 2. The inertia frame is established with the center of mass projection to the ground as the origin according to the right hand rule. The  $\omega_{steer}$  means the angle of handlebar. The  $\omega_{yaw}$  and  $\omega_{pitch}$  mean angles of yaw and pitch of the BR respectively. The  $\dot{\omega}_{roll}$  and  $\dot{\omega}_{rear}$  are angular velocity of roll and rear wheel respectively. The  $\dot{\omega}_{inertia}$  is the angular velocity of the inertia wheel.

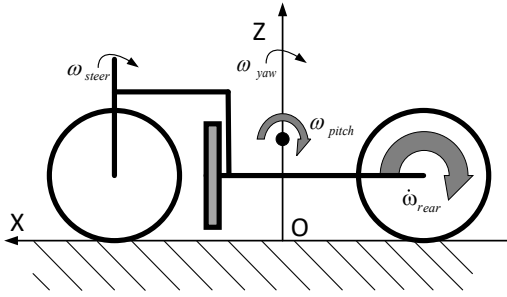


Fig. 1. The side view of BR.

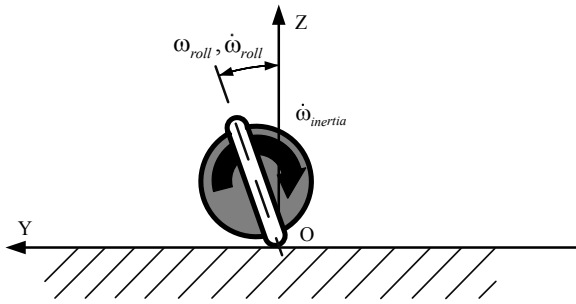


Fig. 2. The back view of BR.

### B. Simulation Environment

In order to quickly verify the algorithm proposed in this paper, an equivalent gravity training environment and a test environment containing rough terrain are constructed in Gazebo. The gravitational acceleration [19] is deflected around the Y-axis as shown in Fig. 3, and the observed states of the BR is processed accordingly. The left one is assumed that the BR is running on the slope where pitching angle is  $\omega_{pitch}$ , yaw angle is  $\omega_{yaw}$ , just as BR is located in plane OBC. In state variables of the robot, pitching angle can be expressed  $\omega_{pitch}^* = \sin^{-1}(\sin(\omega_{pitch}) \times \cos(\omega_{yaw}))$ . It can

be seen in the right part of Fig. 3, virtual gravitational field is formed by gravitational acceleration vector rotating the angle of  $\omega_{pitch}$  around Y axis. The state of low-level controller is  $s_l^* = (\omega_{roll}, \omega_{pitch}^*, \omega_{yaw}, \dot{\omega}_{roll}, \omega_{steer}, \dot{\omega}_{rear})$ .

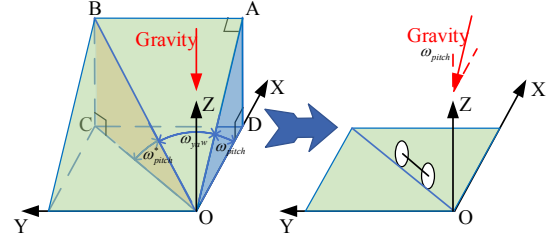


Fig. 3. The structure of equivalent gravity simulation environment.

As for the rough terrain environment for the test of the controller, this paper takes the grayscale image of a winding mountain road in an area from Google Maps, and then uses pixel brightness as Gazebo's terrain altitude to generate the 3D solid model (Fig. 4) [20]. Several waypoints are set near the center line of the mountain road as the path tracking targets of the robot. This path has some bump disturbance and a changing pitch angle for BR.

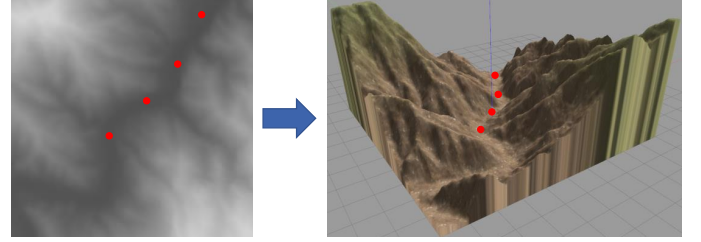


Fig. 4. The grayscale image and 3D model of the target terrain.

## III. METHODS

### A. The Two-level Controller

The proposed controller in this paper is shown in Fig. 5. The low-level controller is trained by proximal policy optimization (PPO) [21]. The high-level controller is trained by PPO with Long Short-Term Memory (LSTM) [22]. The specific input of each module are listed in the next section. The low-level controller consists of the agent pool and the policy selection module, which is simply represented as the low-level policy network. The low-level controller trained in the equivalent gravity environment and gives the angular velocity command of the inertia wheel  $a_i^*$ . The Stanley module gives the steering angle command  $a_s^*$ . The high-level policy network module is called residual reinforcement learning. The output of the high-level policy network is used to compensate the gap between the training environment and the target environment. The final output of the controller is  $[a_r, a_s, a_i]$ , where  $a_r$  is the angular velocity command of the rear wheel.

$$\begin{bmatrix} a_r \\ a_s \\ a_i \end{bmatrix} = \begin{bmatrix} a_r \\ a_s^* + \Delta a_s \\ a_i^* + \Delta a_i \end{bmatrix} = \pi_\theta(S_{h0}) + \begin{bmatrix} 0 \\ \pi_s(S_{h1}) \\ \pi_\rho(S_l) \end{bmatrix} \quad (1)$$

where  $\pi_\theta$  is the high-level policy network,  $\pi_s$  means the controller of Stanley,  $\pi_\rho$  is the low-level policy network.

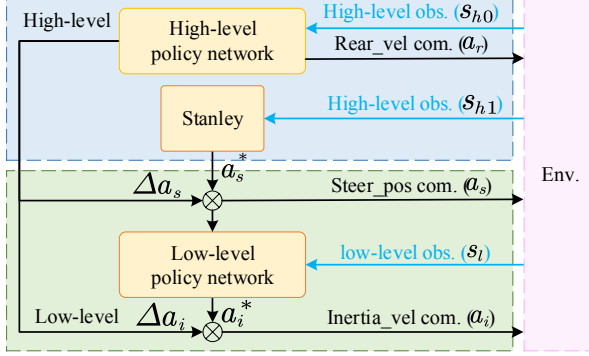


Fig. 5. The two-level controller.

### B. PCL for Balancing Control

The PCL is proposed to order the training sequence of balancing control tasks, as shown in Fig. 6. Different from the method in [18], this paper focus on the following two points: i) The target strategy is not the last agent obtained at the end of all courses in PCL, but the agent pool formed through agents and course performance continuously in the training process. ii) We used the average reward value of the test task to order the courses. Such modifications also facilitate understanding of agent performance on different tasks. That is, each task  $T_i$  corresponds to a subset of the state space. A task proposal module determines the current training task according to agent's performance in different tasks. The details can be found in Algorithm 1.

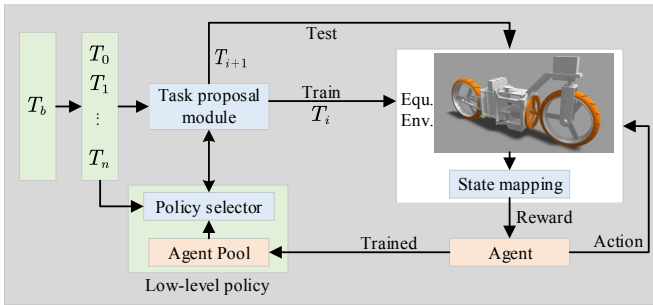


Fig. 6. The piecewise curriculum learning for balancing control.

**Require:** State subspace  $S_i (i = 1, \dots, n)$ , and corresponding tasks  $T_i (i = 1, \dots, n)$ , policy pool  $P$ , policy selection module  $M$ , the threshold of reward  $R_t$ , the maximum number of sub-task training  $K$ .

Initialization:  $i = 1, j = 1$

**repeat**

**repeat**

Sample initial state  $s$  from  $S_i$

$\pi_i \leftarrow \pi_{i-1}$

Train policy  $\pi_i$  for task  $T_i$

Get the average reward  $R'$  on  $S_i$

$j \leftarrow j + 1$

**until**  $j > K$  or  $R' > R_t$

Store  $\pi_i$  in  $P$

Store  $(S_i, \pi_i)$  pair in  $M$

**repeat**

Sample initial state  $s$  from  $S_{i+1}$

Test  $\pi_i$  on  $S_{i+1}$  and get the average reward  $R'$

$\pi_{i+1} \leftarrow \pi_i$

Store  $(S_{i+1}, \pi_{i+1})$  pair in  $M$

$i \leftarrow i + 1$

**until**  $R' < R_t$  or  $i > n$

$j = 1$

$i \leftarrow i + 1$

**until**  $i > n$

## IV. TASKS AND RESULTS

### A. Experimental Setup

Task configurations of balancing control and path tracking are shown in Table I. The state  $S_l = \{\omega_{roll}, \omega_{pitch}, \omega_{yaw}, \dot{\omega}_{roll}, \dot{\omega}_{steer}, \dot{\omega}_{rear}\}$  is the measurement of the robot's attitude angle and partial angular velocity. The state  $S_{h1} = \{\omega_{yaw}, \omega_{steer}, \dot{\omega}_{rear}, o_x, o_y, p_t^x, p_t^y, p_t^{yaw}\}$  is the input of the Stanley module, which gives an original command of steering angle. The state  $S_{h0} = \{\omega_{roll}, \omega_{pitch}, \omega_{yaw}, \dot{\omega}_{roll}, \dot{\omega}_{steer}, \dot{\omega}_{rear}, v_x, v_y, o_x, o_y, p_t^x, p_t^y, p_t^{yaw}, p_{t+1}^x, p_{t+1}^y, p_{t+1}^{yaw}\}$  not only contains the robot's attitude information and path tracking information, but also the information of the next path reference point for the calculation of the high-level reward. The hyper-parameters of algorithms and network are given in Table II. The reward function  $R_{low}$  is used in balance control task and is shown in (2).

TABLE I  
TASK CONFIGURATION.

|            | Name            | Value                                      |
|------------|-----------------|--------------------------------------------|
| Low-level  | States          | $S_l$                                      |
|            | Actions         | $[a_s^*]$                                  |
|            | Reward function | $R_{low}$                                  |
|            | End condition   | $ \omega_{roll}  > 0.6rad$                 |
| High-level | States          | $[S_{h0}, S_{h1}]$                         |
|            | Actions         | $[a_r, a_s, \Delta a_i]$                   |
|            | Reward function | $R_{high}$                                 |
|            | End condition   | $ \omega_{roll}  > 0.6rad$ or $d_t > 0.5m$ |

$$R_{low} = \min[\cos(2\dot{\omega}_{roll}^2 - 1), 3.0] \quad (2)$$

The high-level reward function that based on potential reward [23]  $R_{high}$  is shown in (3)-(7).  $R_t^*$  is the product of a function of the projection speed of the line between  $p_t$  and

TABLE II  
HYPER-PARAMETERS AND CONFIGURATIONS OF ALGORITHMS.

|                   | PPO       | PPO+LSTM  |
|-------------------|-----------|-----------|
| Buffer size       | 600       | 1000      |
| Batch size        | 64        | 256       |
| Max episodes      | 10000     | 10000     |
| Max step          | 100       | 100       |
| Gamma             | 0.99      | 0.99      |
| Learning rate     | $3e-4$    | $3e-4$    |
| Actor network     |           |           |
| First layer:      | 128 units | 128 units |
| Second layer:     | 128 units | 128 units |
| Activation layer: | tanh      | tanh      |
| Critic network    |           |           |
| First layer:      | 128 units | 128 units |
| Second layer:     | 128 units | 128 units |
| Activation layer: | tanh      | tanh      |
| GRU               |           |           |
| First layer:      | -         | 128 units |
| Sequence length   | -         | 4         |

$p_{t+1}$  of centroid speed and a function of the distance between  $p_t$  and centroid of the robot in Fig. 7.

$$\alpha_t = \text{atan}\left(\frac{p_{t+1}^y - p_t^y}{p_{t+1}^x - p_t^x}\right) \quad (3)$$

$$v_t = v_x \cos \alpha_t + v_y \sin \alpha_t \quad (4)$$

$$d_t = \sqrt{(o_x - p_t^x)^2 + (o_y - p_t^y)^2} \quad (5)$$

$$R_t^* = \min[v_t^2, 3] \cdot \min[\text{acos}(2d_t^2 - 1), 3.0] \quad (6)$$

$$R_{high} = 10(R_t^* - R_{t-1}^*) \quad (7)$$

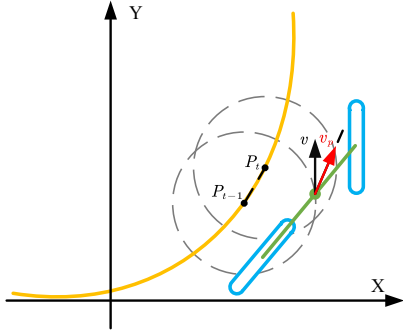


Fig. 7. The task of path tracking.

### B. Balancing control

The state space  $S_b$  is a 3-dimension space constructed by the pitching angle  $\omega_{pitch} \in \{-0.15\text{rad}, 0.15\text{rad}\}$ , steering angle  $\omega_{steer} \in \{-0.5\text{rad}, 0.5\text{rad}\}$ , and the range of angular velocity of rear wheel for the BR  $0.0\text{rad/s}, 20.0\text{rad/s}$ , in which the normalized  $\omega_{rear} \in \{0.0, 1.0\}$ . For convenience,  $\psi, \theta, \omega$  are used to represent  $\omega_{pitch}, \omega_{steer}, \omega_{rear}$  respectively below.

$$S_b = \{(\psi, \theta, \omega) \mid -0.15 \leq \psi \leq 0.15, -0.5 \leq \theta \leq 0.5, 0 \leq \omega \leq 1\} \quad (8)$$

The original subspace  $S_0$ :

$$S_0 = \{(\psi, \theta, \omega) \mid -0.15p_0 \leq \psi \leq 0.15p_0, -0.5p_0 \leq \theta \leq 0.5p_0, 0 \leq \omega \leq p_0\} \quad (9)$$

where  $p_0$  is 0.1. A ratio  $p_n$  is  $0.1 + 0.02n$ , and different subtask spaces  $S_n$  will be constructed.

$$S_1^* = \{(\psi, \theta, \omega) \mid -0.15p_1 \leq \psi \leq 0.15p_1, -0.5p_1 \leq \theta \leq 0.5p_1, 0 \leq \omega \leq p_1\} \quad (10)$$

$$S_1 = S_1^* - S_0 \quad (11)$$

$$S_n^* = \{(\psi, \theta, \omega) \mid -0.15p_n \leq \psi \leq 0.15p_n, -0.5p_n \leq \theta \leq 0.5p_n, 0 \leq \omega \leq p_n\} \quad (12)$$

$$S_{n+1} = S_{n+1}^* - S_n^*, 1 \leq n \leq 44 \quad (13)$$

$T_i$  represents the balance control task under the state of the velocity, steering angle and pitch angle sampled on  $S_i$ . The training process is organized according to algorithm 1, and  $R_t = 250$ ,  $I = 100$ . Originally, sampling of  $(\psi, \theta, \omega)$  will be done by the agent in  $S_0$  rather than in  $S_b$ . In running process of every episode,  $(\psi, \theta, \omega)$  remains the same. When performance of the agent  $\pi$  is greater than the given threshold  $R_t$ , the sampling area of the agent will transfer to the next area and saved, then training will start from the next region.

Training results can be seen in Fig. 8 and Fig. 9, the horizontal axis means the iterations of neural network policy parameters, and the vertical axis represents the mean reward in corresponding task area. Training starts from  $T_0$  (i.e. percent = 0.1), and the reward becomes greater than  $R_t$  in the 25th iteration of PPO parameters. The next task becomes  $T_1$  (i.e. percent = 0.12). The  $\pi_{25}$  is used for iteration in  $T_1$  and the reward is still greater than  $R_t$ . The reward decreases sharply when  $m$  is 29. Therefore,  $\pi_{29}$  can be used in the task space where percent is less than 0.4. Similarly, the training results are shown in Table 3, and the change of percent can be found in Fig. 8. As the policy training continued, the ideal training performance is achieved when  $p < 0.8$ . State space can be divided into four regions:

$$D_0 = \bigcup_{n=0}^{15} S_n, D_1 = \bigcup_{n=16}^{30} S_n, D_2 = \bigcup_{n=31}^{35} S_n, D_3 = \bigcup_{n=36}^{45} S_n \quad (14)$$

Exploration of angular velocity of rear wheel and steering angle is limited in region  $D_0, D_1$  and  $D_2$  according to pitching angle of the road during high-level training, and the piecewise policies are:

$$\pi = \begin{cases} \pi_{29}, & \text{if } s^* \in D_0 \\ \pi_{76}, & \text{if } s^* \in D_1 \\ \pi_{235}, & \text{if } s^* \in D_2 \end{cases} \quad (15)$$

where  $s^* = (\omega_{pitch}^*, \omega_{steer}^*, \omega_{rear}^*)$ . Finally, multiple agents and their total corresponding balance control regions are:  $S_{35}^* = \{(\psi, \theta, \omega) \mid -0.15p_n \leq \psi \leq 0.15p_n, -0.5p_n \leq \theta \leq 0.5p_n, 0 \leq \omega \leq 20p_n\}$ ,  $p_{35} = 0.1 + 0.02 \times 35 = 0.8$ . According to algorithms mentioned above and the relation between agent and percent of Table 3, the results of testing

in  $S_{35}^*$  are shown in Fig. 10, and the region in this figure can be divided into three parts including  $D0, D1, D2$  based on piecewise policies. The curriculum RL [18] is tested and is shown in Fig. 10. According to the setting of reward function in 5.1, the maximum reward in one episode is 300. Therefore, it shows that the proposed PCL algorithm achieves better results than curriculum RL.

TABLE III  
CORRESPONDENCE BETWEEN TASK SPACE AND AGENT.

| Percent (p) | [0, 0.4] | [0.42, 0.7] | [0.72, 0.8] | [0.82, 1] |
|-------------|----------|-------------|-------------|-----------|
| Agent       | 29       | 76          | 235         | -         |
| Region      | D0       | D1          | D2          | D3        |

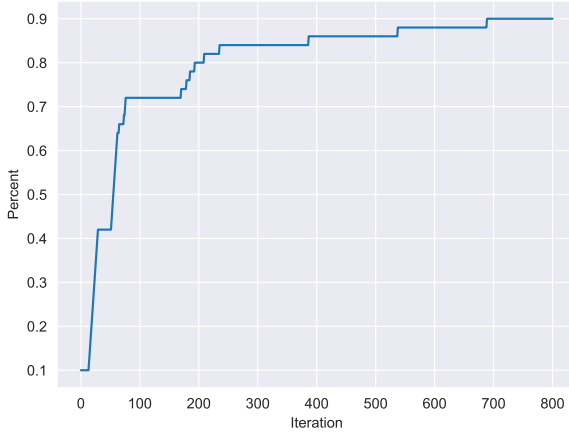


Fig. 8. Percent of low-level training.

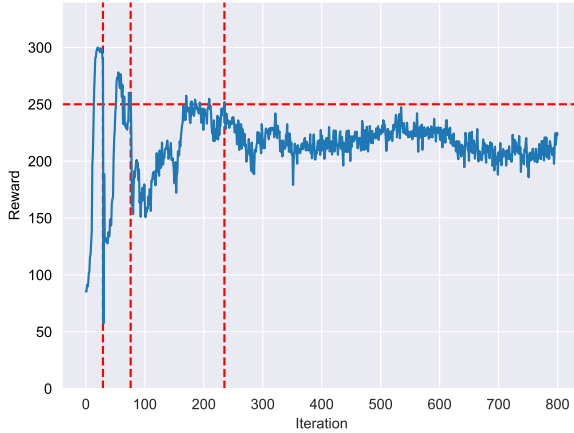


Fig. 9. Iterative training process of low-level policy.

### C. RRL

The training process of high-level policy is shown in Fig. 11, reward of every episodes is increasing and becoming convergent when after episode 600 during training process. The state



Fig. 10. Test of low-level policy.

trajectories in the initial stage and at the completion of RRL training are shown in Fig. 12 and Fig. 13 respectively. With the compensation of RRL, the time required for BR to reach the destination is reduced. And the roll angle error and the path tracking error are also significantly reduced. All of these indicate that the controller based on reinforcement learning proposed in this paper has a good effect.

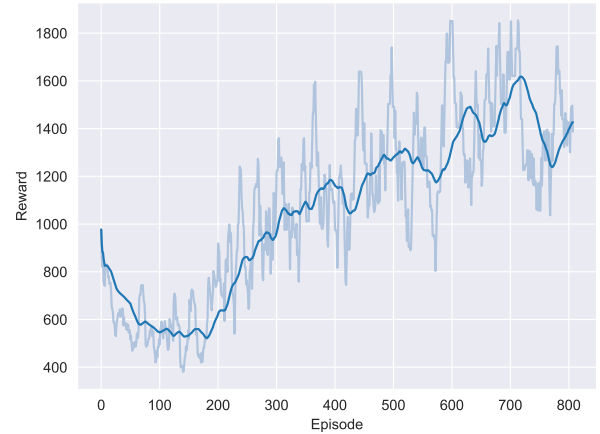


Fig. 11. The training process of high-level policy.

## V. CONCLUSIONS

This paper proposes a BR motion controller based on reinforcement learning. The controller is used to achieve BR balance control and path tracking control on rough terrain. The main contribution of this paper are as follows: 1) This paper is different from existing related studies about the BR from the perspective of the task. Previous studies have not considered the influence of an unstructured rough terrain on the motion control of BR. 2) This paper puts forward



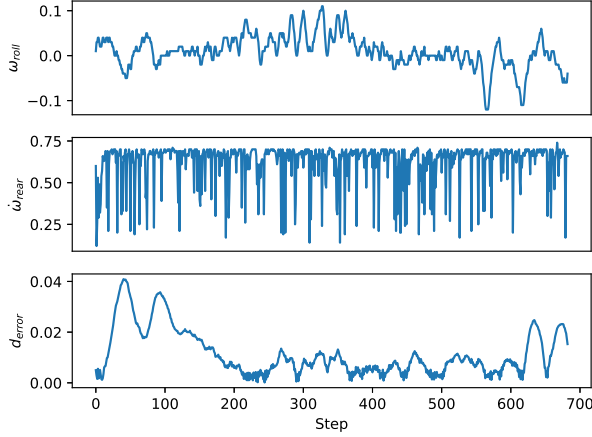


Fig. 12. The state trajectories without compensation. (1 step = 0.02s)

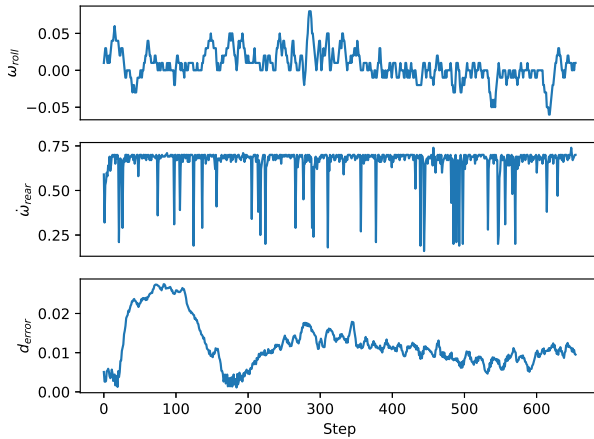


Fig. 13. The state trajectories with compensation. (1 step = 0.02s).

the method of reinforcement learning based on segmented curriculum. The method of sectional curriculum reinforcement learning is compared with that of traditional curriculum reinforcement learning, which shows better effect. In addition, in the aspect of path tracking control, this paper chooses Stanley algorithm as the basis. Then, RRL-based compensation for balance control and path tracking control is presented. By comparison, the improvement of RRL control effect on BR is illustrated.

The following problems still exist in this paper, which will be considered in the future work. Firstly, adding more comparison tests, including algorithms of the latest course learning baseline, different environments comparing BR controls on flat and rough terrain, various data-driven terrains with thousands of different shapes and roughness, to prove the effectiveness of the controller proposed in this paper. Furthermore, in order to further improve the practicality of DRLs in BR, Gaussian process-based RRLs will be considered in future work to implement RRLs online compensation.

## REFERENCES

- [1] Malizia, F., Blocken, B. Bicycle aerodynamics History, state-of-the-art and future perspectives. *Journal of Wind Engineering and Industrial Aerodynamics*, 200, 104134 (2020).
- [2] Jones, D. E.: The stability of the bicycle. *Physics Today*, 23, 34-40 (1970).
- [3] Getz, N. H.: Control of balance for a nonlinear nonholonomic non-minimum phase model of a bicycle. *Proceedings of 1994 American Control Conference - ACC '94*, 1, 148-151 vol.1 (1994).
- [4] Meijaard, J.P., Papadopoulos, J., Ruina, A., & Schwab, A.L.: Linearized dynamics equations for the balance and steer of a bicycle: a benchmark and review. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 463, 1955 - 1982 (2007).
- [5] Hwang, C., Wu, H., & Shih, C.: Fuzzy Sliding-Mode Underactuated Control for Autonomous Dynamic Balance of an Electrical Bicycle. *IEEE Transactions on Control Systems Technology*, 17, 658-670 (2009).
- [6] Kooijman, J., Meijaard, J.P., Papadopoulos, J., Ruina, A., & Schwab, A.L.: A Bicycle Can Be Self-Stable Without Gyroscopic or Caster Effects. *Science*, 332, 339 - 342 (2011).
- [7] Xiong, J., Wang, N., & Liu, C.: Stability analysis for the Whipple bicycle dynamics. *Multibody System Dynamics*, 48, 311-335 (2019).
- [8] Keo, L., & Masaki, Y.: Trajectory control for an autonomous bicycle with balancer. *2008 IEEE/ASME International Conference on Advanced Intelligent Mechatronics*, 676-681 (2008).
- [9] Persson, N., Ekström, M.C., Ekström, M., & Papadopoulos, A.V.: Trajectory tracking and stabilisation of a riderless bicycle\*. *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, 1859-1866 (2021).
- [10] Cossalter V, Lot R, Massaro M.: *Motorcycle Dynamics* (2014).
- [11] Sutton, R.S., & Barto, A.G.: *Reinforcement Learning: An Introduction*. *IEEE Transactions on Neural Networks*, 16, 285-286 (2005).
- [12] Arulkumaran, K., Deisenroth, M.P., Brundage, M., & Bharath, A.A.: *Deep Reinforcement Learning: A Brief Survey*. *IEEE Signal Processing Magazine*, 34, 26-38 (2017).
- [13] Randløv, J., & Alström, P. *Learning to Drive a Bicycle Using Reinforcement Learning and Shaping*. *ICML* (1998).
- [14] Tuyen, L.P., & Chung, T. Controlling bicycle using deep deterministic policy gradient algorithm. *2017 14th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI)*, 413-417 (2017).
- [15] Zhu, X., Zheng, X., Zhang, Q., Chen, Z., Liu, Y., & Liang, B. Natural Residual Reinforcement Learning for Bicycle Robot Control. *2021 IEEE International Conference on Mechatronics and Automation (ICMA)*, 1201-1206 (2021).
- [16] Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M.E., & Stone, P.: Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey. *J. Mach. Learn. Res.*, 21, 181:1-181:50 (2020).
- [17] Portelas, R., Colas, C., Weng, L., Hofmann, K., & Oudeyer, P. Automatic Curriculum Learning For Deep RL: A Short Survey (2020). *ArXiv*, abs/2003.04664.
- [18] Zhang, Y., Abbeel, P., & Pinto, L. Automatic Curriculum Learning through Value Disagreement (2020). *ArXiv*, abs/2006.09641.
- [19] Zhang, F., Yu, Y., Wang, Q., & Zeng, X.: Physics-driven locomotion planning method for a planar closed-loop terrain-adaptive robot. *Mechanism and Machine Theory*, 162, 104353 (2021).
- [20] Abbyasov, B., Lavrenov, R., Zakiev, A., Yakovlev, K.S., Svinin, M.M., & Magid, E.: Automatic tool for Gazebo world construction: from a grayscale image to a 3D solid model. *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 7226-7232 (2020).
- [21] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O.: Proximal Policy Optimization Algorithms (2017). *ArXiv*, abs/1707.06347.
- [22] Greff, K., Srivastava, R.K., Koutník, J., Steunebrink, B.R., & Schmidhuber, J.: LSTM: A Search Space Odyssey. *IEEE Transactions on Neural Networks and Learning Systems*, 28, 2222-2232 (2017).
- [23] Ng, A., Harada, D., & Russell, S.J.: Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. *ICML* (1999).